



Finding Introverted Cores in Bipartite Graphs

Kaiyuan Shu¹, Qi Liang¹, Haicheng Guo¹, Fan Zhang^{1(✉)}, Kai Wang²,
and Long Yuan³

¹ Guangzhou University, Guangzhou 510006, China
kaiyuanshu@e.gzhu.edu.cn, zhangf@gzhu.edu.cn

² Shanghai Jiao Tong University, Shanghai 200240, China

³ Nanjing University of Science and Technology, Nanjing 210094, China

Abstract. In this paper, we propose a novel cohesive subgraph model to find introverted communities in bipartite graphs, named (α, β, p) -core. It is a maximal subgraph in which every vertex in one part has at least α neighbors and at least p fraction of its neighbors in the subgraph, and every vertex in the other part has at least β neighbors. Compared to the (α, β) -core, the additional requirement on the fraction of neighbors in our model ensures that the vertices are introverted in the subgraph regarding their neighbor sets, e.g., for the (α, β) -core of a customer-product network, the shopping interests of the customers inside focus on the products in the subgraph. We propose an $O(m)$ algorithm to compute the (α, β, p) -core with given α, β and p . Besides, we introduce an efficient algorithm to decompose a graph by the (α, β, p) -core. The experiments on real-world data demonstrate that our model is effective and our proposed algorithms are efficient.

Keywords: Cohesive subgraph · Introverted community · Bipartite graph

1 Introduction

Many real-world relationships among various entities are modeled as bipartite graphs, such as collaboration networks [11], gene co-expressions networks [9], customer-product networks [19], and user-page networks [4]. The analysis of bipartite graphs become increasingly important with well applied, makes (α, β) -core a valuable model in mining cohesive subgraphs with applications mainly in [5, 7, 20, 23]. The (α, β) -core of G is a maximal bipartite subgraph G' with two node sets $U' \subseteq U$ and $V' \subseteq V$ and every vertex has at least α and β neighbors. Although it can ensure certain engagement of vertices, the characteristic of vertices would be ignored as vertices have many neighbors outside the subgraph.

Motivated by above, we propose (α, β, p) -core as a novel cohesive subgraph model. It is a maximal subgraph G' with two node sets $U' \subseteq U$ and $V' \subseteq V$, where vertices in U' have at least α neighbors and at least p fraction of its

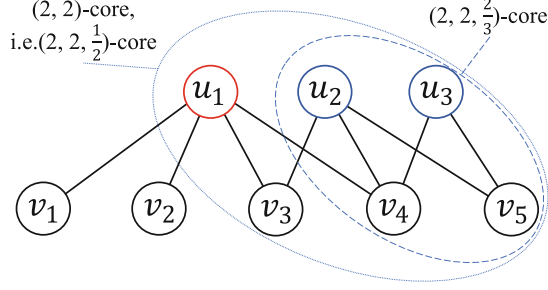


Fig. 1. An Instance of (α, β, p) -core

neighbors, with degree of vertices in V' is at least β in subgraph. The fraction constraint p is to capture introverted users and explore finer granularity in characterizing users accurately with (α, β) -core refined. Figure 1 shows an instance of (α, β, p) -core. The $(2, 2)$ -core induced by u_1 to u_3 and v_3 to v_5 also a $(2, 2, \frac{1}{2})$ -core. $(2, 2, \frac{2}{3})$ -core is induced by u_2, u_3, v_4, v_5 , as u_2, u_3 are more introverted and homogeneous while the focus of u_1 is not well covered in the core.

In the following, we summarize the contributions of this paper.

- We propose a novel cohesive subgraph (α, β, p) -core for bipartite graphs. With the extra fraction constraint p , it can customize the degree requirement for every vertex and capture introverted vertices compared to the (α, β) -core.
- We devise an algorithm with $O(m)$ time complexity to compute a (α, β, p) -core subgraph with given α, β and p values. We propose a decomposition algorithm with $O(dmax_U(G) \cdot dmax_V(G) \cdot m)$ time cost to compute (α, β, p) -core for all α, β and p values, where $dmax_U(G)$ (resp. $dmax_V(G)$) is the maximum degree of vertices in set $U(G)$ (resp. $V(G)$), and m is the number of edges in G .
- The experiments on several real-world graphs demonstrate the effectiveness of our (α, β, p) -core model and the efficiency of our algorithms.

2 Related Work

Cohesive subgraph mining is a fundamental problem in graph structure analysis. The fundamental structure of dense subgraph is clique [13], with other new structures like quasi-clique [1], k -plex [17], k -truss [18] and k -core [16]. [3] gives an $O(m)$ algorithm for core decomposition. [22] proposes a finer granularity model and corresponding efficient algorithms. They are all proposed for general graphs. For cohesive subgraphs mining in bipartite graphs, [8] and [10] propose the definition of biclique on bipartite graphs. [24] proposes an efficient algorithm to enumerate all the bicliques. [15] defines a framework of bipartite subgraphs based on the butterfly motif ($2, 2$ -biclique) to model the dense regions. [14] studies the problem of butterfly counting. The (α, β) -core was proposed by [2], [6] and [7]

proposed the linear algorithm to compute (α, β) -core. An index-based method computation of (α, β) -core is proposed in [12]. [21] studies the problem of (α, β) -core decomposition. To the best of our knowledge, we are the first to consider the fraction constraint of vertex neighbors in bipartite subgraphs.

3 Preliminaries

The bipartite graph $G = (U, V, E)$ has two node sets U and V , with edges from $E \subseteq U \times V$. $U(G)$ and n_u (resp. $V(G)$ and n_v) are the node set and node number of G , with n as the total number of nodes. $E(G)$ and m are edge set and total edges numbers of G . The degree of $u \in U(G) \cup V(G)$ is $\deg(u, G)$, with maximum degree $\text{dmax}_U(G)$ (resp. $\text{dmax}_V(G)$) in $U(G)$ (resp. $V(G)$). For subgraph G' of G with node sets $U' \subseteq U$ and $V' \subseteq V$, we denote $U(G') = U'$, $V(G') = V'$ and $E(G') = E(G) \cap (U' \times V')$. To simplify, we represent bipartite graphs as graphs.

Definition 1 ((α, β)-core). Given a graph G and two integers α and β , a subgraph S is the (α, β) -core of G , denote by $C_{\alpha, \beta}(G)$, if (1) $\deg(u, S) \geq \alpha$ for every $u \in U(S)$; (2) $\deg(v, S) \geq \beta$ for every $v \in V(S)$; (3) S is maximal, i.e., any subgraph S' is not a (α, β) -core if S is a subgraph of S' and $S \neq S'$.

Obviously, the (α, β) -core of G is unique with given α and β . According to Definition 1, the (α, β) -core of G can be computed by removing vertices which not satisfy the degree constraint with time complexity $O(m)$ in the worst case.

Definition 2 (α/β -core number). Given an integer α , if a vertex $w \in G$ can be in an (α, β) -core but not in $(\alpha, \beta + 1)$ -core, then we say the α -core number of node w is β , denote as $cn_\alpha(w) = \beta$. Correspondingly, the β -core number of w is defined when β is fixed and we denote it by $cn_\beta(w)$.

From Definition 2, whether a vertex belongs to a (α, β) -core relies on its α (or β)-core number. Since the α -core number and β -core number are similar, we focus on α -core number for the rest. Before introducing the (α, β, p) -core, we define the fraction of a vertex regarding a subgraph S and the graph G .

Definition 3 (fraction). Given a graph G and a subgraph S of G , the fraction of vertex v in S is defined as the degree of v in S divided by the degree of v in G , i.e., $\text{frac}(v, S, G) = \deg(v, S) / \deg(v, G)$.

Based on the Definition 2 and 3, we define (α, β, p) -core as follows.

Definition 4 ((α, β, p)-core). Given a graph G , two integers α and β , and a decimal p , a subgraph S is the (α, β, p) -core of G , denoted by $C_{\alpha, \beta, p}(G)$, if every vertex u in $U(G)$ and v in $V(G)$, (1) $\deg(u, S) \geq \alpha$ and $\deg(v, S) \geq \beta$ (degree constraint); (2) $\text{frac}(u, S, G) \geq p$ (fraction constraint) and (3) S is maximal, i.e., any subgraph S' is not a (α, β, p) -core if S is a subgraph of S' and $S \neq S'$.

Example 1. In Fig. 1, the $(2, 2)$ -core have vertices u_1 to u_3 and v_3 to v_5 . The 2-core number of vertex u_1 is 2, i.e., $cn_2(u_1) = 2$. As two neighbors of u_1 (i.e., v_1 and v_2) are not in the $(2, 2)$ -core, the fraction of u_1 in $(2, 2)$ -core is 0.5. The $(2, 2)$ -core is a $(2, 2, \frac{1}{2})$ -core which also contains $(2, 2, \frac{2}{3})$ -core.

Algorithm 1: (α, β, p) -core computation

Input: a graph G , the degree threshold α, β , the fraction threshold p
Output: (α, β, p) -core of G

```

1  $t[u] \leftarrow \max(\alpha, \lceil p \times \deg(u, G) \rceil)$  for every vertex  $u \in U(G)$ ;
2  $t[v] \leftarrow \beta$  for every vertex  $v \in V(G)$ ;
3  $G' \leftarrow G$ ;
4 while  $\exists u \in U(G') : \deg(u, G') < t[u]$  or  $\exists v \in V(G') : \deg(v, G') < t[v]$  do
5    $\quad$  remove  $u$  (or  $v$ ) and its incident edges from  $G'$ ;
6 return  $G'$ ;
```

Algorithm 2: p -number computation

Input: a (α, β) -core of G named S
Output: the p -core numbers of all vertices in S

```

1  $S \leftarrow (\alpha, \beta)$ -core of  $G$ ;
2 while  $U(S)$  is not empty do
3    $p_{min} \leftarrow \min\{\deg(u, S)/\deg(u, G), \forall u \in U(S)\}$ ;
4   while  $\exists u \in U(S) : \deg(u, S) < \alpha$  or  $\deg(u, S)/\deg(u, G) \leq p_{min}$  do
5      $p_{(\alpha, \beta)}[u] \leftarrow p_{min}$ ;
6     remove  $u$  and its incident edges from  $S$ ;
7     while  $\exists v \in V(S) : \deg(v, S) < \beta$  do
8        $p_{(\alpha, \beta)}[v] \leftarrow p_{min}$ ;
9       remove  $v$  and its incident edges from  $S$ ;
10 return  $p_{(\alpha, \beta)}[w]$ ;
```

With any α, β and p , the (α, β, p) -core always be a subgraph of (α, β) -core in G , and unique as it is required to be maximal when α, β and p are fixed. In the subsequent sections, we study the (α, β, p) -core computation in two parts: (1) we compute the (α, β, p) -core of G with specified α, β and p values; and (2) the (α, β, p) -core decomposition retrieves (α, β, p) -core for all possible triples (α, β, p) values where $1 \leq \alpha \leq \max_U(G), 1 \leq \beta \leq \max_V(G)$ and $0 \leq p \leq 1$.

4 (α, β, p) -Core Computation and Decomposition

Algorithm 1 shows the pseudo-code of (α, β, p) -core computation with given α, β , and p , with removing vertices iteratively which not satisfy the constraint until no nodes can be removed. We compute the degree threshold in $U(G)$ as the larger one of α and $\lceil p \times \deg(u, G) \rceil$, with degree threshold β of $V(G)$ in Lines 1–2. We check the degree on G in Line 3. If u and v cannot satisfy the degree threshold, we delete them and their incident edges (Line 4). Note that the deletion of a vertex can decrease the degrees of its neighbors, we should delete the vertex and its adjacent edges when its degree is smaller than the degree threshold. We do not need to update the vertex degree which is already under the degree threshold.

Example 2. Figure 1 shows an example graph. For $(2, 2, \frac{2}{3})$ -core, we compute the degree threshold for every vertex at first. As u_1, u_2 and u_3 have thresholds $\frac{8}{3}, 2$, and $\frac{4}{3}$, the threshold in $V(G)$ is 2. We delete u_1, v_1 , and v_2 and their incident edges since they cannot satisfy the threshold. As the remaining nodes satisfy the degree and fraction constraint, we return the subgraph of u_2, u_3, v_3, v_4 , and v_5 .

Algorithm 3: (α, β, p) -core decomposition

Input: a graph G
Output: the (α, β, p) -core numbers of all vertices in G

```

1 foreach  $\alpha = 1$  to  $dmax_U(G)$  do
2    $G' \leftarrow G$ ;
3   while  $\exists u \in U(G')$  and  $deg(u, G') < \alpha$  do
4      $\perp$  remove  $u$  and its incident edges from  $G'$ ;
5   while  $G'$  is not empty do
6      $\beta_{min} \leftarrow \min\{deg(v, G') \mid v \in V(G')\}$ ;
7     do  $p$ -number computation on  $G'$  ( $(\alpha, \beta_{min})$ -core);
8     while  $\exists v \in V(G')$  and  $deg(v, G') \leq \beta_{min}$  do
9       remove  $v$  and its incident edges from  $G'$ ;
10    while  $\exists u \in U(G')$  and  $deg(u, G') < \alpha$  do
11       $\perp$  remove  $u$  and its incident edges from  $G'$ ;

```

12 **return** the (α, β, p) -core numbers of all vertices in G ;

Although the α (resp. β) ranges from 1 to $dmax_U(G)$ (resp. $dmax_V(G)$), p ranges from 0 to 1 continuously, many (α, β, p) -cores are the same with different α, β or p . Based on this observation, we first define the p -number of a vertex.

Definition 5 (p -number). *Given a graph G and degree thresholds α, β , decimal p is the p -number of a vertex u , denoted by $pn(u, \alpha, \beta, p)$, if (1) the (α, β, p) -core contains u and (2) the (α, β, p') -core does not contain u for any $p' > p$.*

Definition 6 ((α, β, p) -core number). *Given a graph G , a (α, β, p) -core number of a vertex is a tuple of $\langle \alpha, \beta, p \rangle$, where the value of p is the p -number of the vertex based on the pair of α and β .*

As above mentioned, we compute the (α, β, p) -core in given α, β , and p in $O(m)$ time. However, Algorithm 1 costs too much to return the result with lots of requests of computing (α, β, p) -core in a large graph. So we introduce the (α, β, p) -core decomposition, which aims to find all (α, β, p) -core numbers and make computation more effective with the result of (α, β, p) -core decomposition.

We propose Algorithm 2 to compute the p -number of given (α, β) -core (S for convenience). The fraction of vertices in S may be different. We compute the minimum fraction for all vertices in S first (Line 3). For vertices in $U(S)$ with a degree smaller than α or fraction not larger than p_{min} , we record the p -number of them as p_{min} (Line 5) and remove them with their incident edges from S at Line 6. As removing nodes in $U(S)$ will decrease the degree of the vertex in $V(G)$, we check the degree of all vertices in $V(S)$. If vertices have a smaller degree than β , we record the p -number of them as p_{min} (Line 8) and remove them with their incident edges in S (Line 9). Repeat these processes until S is empty, then we return the p -number of all vertices in given (α, β) -core.

Algorithm 3 is (α, β, p) -core decomposition. We fix α from 1 to $dmax_U(G)$ (Line 1) and delete vertices with degree smaller than α in $U(G)$, ensured the new subgraph is $(\alpha, 1)$ -core (denoted G') (Lines 3–4). As the minimum degree value (β_{min}) reveal in Line 6, we do the p -number computation on (α, β_{min}) -core and

get tuples of (α, β_{min}, p) as (α, β, p) -core numbers. We delete the vertices with degree is β_{min} in $V(G')$ to ensure the degree of vertices in $U(G')$ is more than α (Lines 8–11). We compute a new minimum degree value and repeat the above process until G' is empty. Finally, we will return a (α, β, p) -core number set of each vertex in G . Note that the computation can be also started by fixing β .

Example 3. In Fig. 1, suppose that we begin the decomposition from $\alpha = 2$. The whole graph is a $(2, 1, 1)$ -core. When $\beta = 2$, v_1 and v_2 would be deleted as they have the smallest degree, and the subgraph is a $(2, 2)$ -core. Node u_1 is the first leaving node as the fraction of u_1 is the smallest, so the (α, β, p) -core number of u_1 is $(2, 2, \frac{1}{2})$. Then v_3 leaves as v_3 does not satisfy the degree threshold β . The u_2 has (α, β, p) -core number $(2, 2, \frac{2}{3})$ since its fraction is the smallest and delete it. After that, the other nodes should be deleted as they do not satisfy the degree constraint, and we finish the decomposition in $\alpha = 2$.

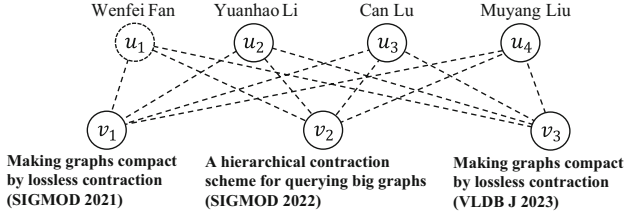


Fig. 2. Case Study on DBLP

Complexity. The time complexity of (α, β, p) -core computation is $O(m)$, with computing degree threshold for each vertex, finding the first vertex to delete after computation, iterative deletion of disqualified vertices while each edge is visited once. The time complexity of p -number computation is $O(m)$. In Algorithm 2, vertex and edge deletion with degree update takes $O(m)$ time as vertices and edges are deleted once. Based on p -number computation, the (α, β, p) -core decomposition costs $O(dmax_U(G) \cdot dmax_V(G) \cdot m)$ time. (Assume $n \ll m$)

5 Experiment

In this section, extensive experiments are conducted to demonstrate the great performance of our model and algorithms.

Datasets. We evaluate the algorithms on five real graphs. The graphs are downloaded from KONECT¹. The details of these graphs are shown in Table 1.

¹ <http://konect.cc/networks/>.

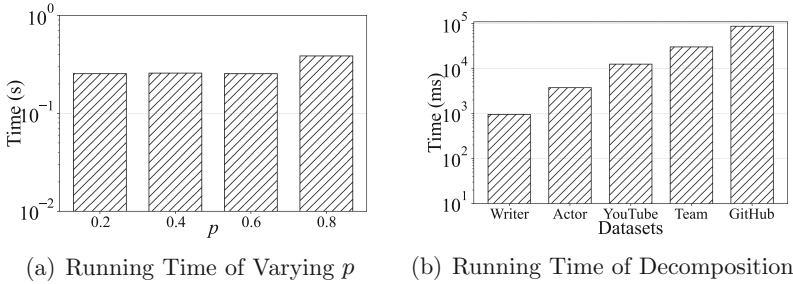
Table 1. Statistics of Datasets

Dataset	Type	$ U $	$ V $	$ E $	$dmax_U$	$dmax_V$
Writers	Authorship	89k	46k	0.14 m	42	246
Actors	Starring	76k	81k	0.28 m	65	321
YouTube	Membership	94k	30k	0.29 m	1,035	7,591
GitHub	Membership	56k	0.12k	0.44 m	884	3,675
Teams	Membership	0.9 m	34k	1.3 m	17	2,671

Parameters. We conduct experiments under different settings by varying the threshold α , β , p . The default values of α , β , and p are 5, 5, and 0.6, respectively.

Environment. We utilize C++ as the programming language for implementing all algorithms and employ GCC (7.5.0) to compile them with O3 optimization. Our experiments are performed on a computer equipped with an Intel Xeon 2.1GHz CPU and 512G of RAM.

Exp-1: Case Study on DBLP. We extract (3, 3, 0.15)-core on DBLP from 2021 to 2023. The chosen subset includes four authors as depicted in Fig. 2. As u_1 to u_4 have 17, 3, 5, and 6 neighbors in (3, 3, 0.15)-core, u_1 is not included in the (3, 3, 0.5)-core due to p constraint. Li, Lu, and Liu focus on graph contraction as their main research area, except Fan, who focuses on graph partitioning, rule discovery, etc. Thus, with the finer structural constraint derived from p , (α, β, p) -core is capable of mining introverted subgraphs compared to the (α, β) -core.

**Fig. 3.** Running Time

Exp-2: Running Time of Varying p . Figure 3(a) shows the computation time of varying p on YouTube with given α and β in 5. The leaving of nodes takes more time as they are smaller than the fraction threshold with p grows. The time-costing deletion proves the remaining graph differ little except $p=0.8$. The vertices count in the final subgraph demonstrates our model can find more homogeneous vertices with p growing from 0 to 1. Overall, the fraction p played an important role in finding introverted and homogeneous vertices of (α, β) -core.

Exp-3: Running Time of Decomposition. Figure 3(b) shows the running time of the decomposition in different datasets. We see that the time of decomposition increases as the size of the dataset grows. The result of the decomposition is in an acceptable range which can be evaluated theoretically in Sect. 4.

6 Conclusion

In this paper, we propose a novel cohesive subgraph model in the bipartite graph, (α, β, p) -core, with an algorithm to compute the (α, β, p) -core in given α , β , and p . Besides, we study and design an algorithm in (α, β, p) -core decomposition.

References

1. Abello, J., Resende, M.G.C., Sudarsky, S.: Massive quasi-clique detection. In: Rajsbaum, S. (ed.) LATIN 2002. LNCS, vol. 2286, pp. 598–612. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-45995-2_51
2. Ahmed, A., Batagelj, V., Fu, X., Hong, S.H., Merrick, D., Mrvar, A.: Visualisation and analysis of the internet movie database. In: 2007 6th International Asia-Pacific Symposium on Visualization, pp. 17–24. IEEE (2007)
3. Batagelj, V., Zaversnik, M.: An $o(m)$ algorithm for cores decomposition of networks. arXiv preprint cs/0310049 (2003)
4. Beutel, A., Xu, W., Guruswami, V., Palow, C., Faloutsos, C.: CopyCatch: stopping group attacks by spotting lockstep behavior in social networks. In: Proceedings of the 22nd International Conference on World Wide Web, pp. 119–130 (2013)
5. Cai, Z., He, Z., Guan, X., Li, Y.: Collective data-sanitization for preventing sensitive information inference attacks in social networks. IEEE Trans. Dependable Secure Comput. **15**(4), 577–590 (2016)
6. Cerinšek, M., Batagelj, V.: Generalized two-mode cores. Soc. Netw. **42**, 80–87 (2015)
7. Ding, D., Li, H., Huang, Z., Mamoulis, N.: Efficient fault-tolerant group recommendation using alpha-beta-core. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, pp. 2047–2050 (2017)
8. Hochbaum, D.S.: Approximating clique and biclique problems. J. Algorithms **29**(1), 174–200 (1998)
9. Kaytue, M., Kuznetsov, S.O., Napoli, A., Duplessis, S.: Mining gene expression data with pattern structures in formal concept analysis. Inf. Sci. **181**(10), 1989–2001 (2011)
10. Lehmann, S., Schwartz, M., Hansen, L.K.: Biclique communities. Phys. Rev. E **78**(1), 016108 (2008)
11. Ley, M.: The DBLP computer science bibliography: evolution, research issues, perspectives. In: Laender, A.H.F., Oliveira, A.L. (eds.) SPIRE 2002. LNCS, vol. 2476, pp. 1–10. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-45735-6_1
12. Liu, B., Yuan, L., Lin, X., Qin, L., Zhang, W., Zhou, J.: Efficient (α, β) -core computation: an index-based approach. In: The World Wide Web Conference, pp. 1130–1141 (2019)
13. Luce, R.D., Perry, A.D.: A method of matrix analysis of group structure. Psychometrika **14**(2), 95–116 (1949)

14. Sanei-Mehri, S.V., Saryüce, A.E., Tirthapura, S.: Butterfly counting in bipartite networks. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 2150–2159 (2018)
15. Saryüce, A.E., Pinar, A.: Peeling bipartite networks for dense subgraph discovery. In: Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining, pp. 504–512 (2018)
16. Seidman, S.B.: Network structure and minimum degree. *Soc. Netw.* **5**(3), 269–287 (1983)
17. Seidman, S.B., Foster, B.L.: A graph-theoretic generalization of the clique concept. *J. Math. Sociol.* **6**(1), 139–154 (1978)
18. Wang, J., Cheng, J.: Truss decomposition in massive networks. arXiv preprint [arXiv:1205.6693](https://arxiv.org/abs/1205.6693) (2012)
19. Wang, J., De Vries, A.P., Reinders, M.J.: Unifying user-based and item-based collaborative filtering approaches by similarity fusion. In: Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 501–508 (2006)
20. Xu, K., Williams, R., Hong, S.-H., Liu, Q., Zhang, J.: Semi-bipartite graph visualization for gene ontology networks. In: Eppstein, D., Gansner, E.R. (eds.) *GD 2009*. LNCS, vol. 5849, pp. 244–255. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-11805-0_24
21. Yu, D., Zhang, L., Luo, Q., Cheng, X., Cai, Z.: Core decomposition and maintenance in bipartite graphs. *Tsinghua Sci. Technol.* **28**(2), 292–309 (2022)
22. Zhang, C., et al.: Exploring finer granularity within the cores: efficient (k, p) -core computation. In: 2020 IEEE 36th International Conference on Data Engineering (ICDE), pp. 181–192. IEEE (2020)
23. Zhang, H., Chen, Y., Li, X., Zhao, X.: Simplifying knowledge-aware aggregation for knowledge graph collaborative filtering. In: Zhao, X., Yang, S., Wang, X., Li, J. (eds.) *WISA 2022*. LNCS, vol. 13579, pp. 52–63. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-20309-1_5
24. Zhang, Y., Phillips, C.A., Rogers, G.L., Baker, E.J., Chesler, E.J., Langston, M.A.: On finding bicliques in bipartite graphs: a novel algorithm and its application to the integration of diverse biological data types. *BMC Bioinform.* **15**, 1–18 (2014)